

Agenda.

- Generics
- Raw Types
- Generic Methods
- Type Erasure

⇒ Use Case.

List of Coordinates. $\Rightarrow (x, y)$
 $\downarrow \quad \swarrow \searrow$
 double double

C++ : Pair
Python : tuple.

Class Fair 1

double first;

double second;

3

```
Pair p = new Pair(-, -);
```

List<pair> Coordinates = _____

⇒ Country, Population

Class Population λ

String country;

long population;

3

Class Pair {

Object first;

Object second;

3

→ Coordinate ?

→ Population ?

Pair p = new Pair(); ⇒ Coordinate

{
p.first = 3.14;
p.second = 4.13;
}

→ p.first = "Scaler"; ⇒ No Compile Time
Error.

double x = (Double) p.getFirst() ⇒ RTE.
"Scaler"

⇒ Run time Exceptions.

Pair p1 = new Pair();

p1.first = "India";

p1.second = — ;

Object : lot of generality but No Compile Time
checks on data types

GENERIC (Java-5).

↳ Placeholder for Data type.

⇒ Compile Time check.

List < X >

ArrayList < X >

HashMap < K, V >

⇒

```
class Pair < T, U > {  
    T first;  
    U second;  
  
    T getfirst() {  
        return this.first;  
    }  
  
    setfirst(T x) {  
        this.first = x;  
    }  
}
```

Pair < String, Double > p = _____
T

List<Car>

#

doSomething (List<Animal> animals) {

=====

3

⇒ doSomething (list of animals); ✓

⇒ doSomething (list of dogs);

Animal a = new Dog(); ✓

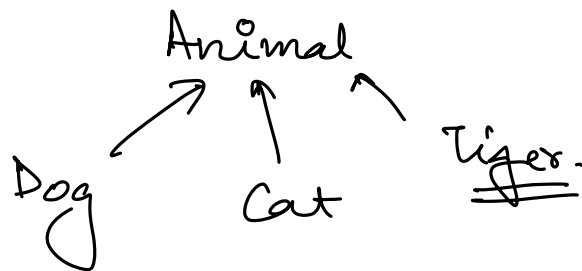
List<Animal> animals = List<Dog> dogs ✗

Animal
↑
~~Dog~~

List<Animal>
↑
List<Dog> ✗

List<Animal> animals = new List<Dog>(); X

List<Animal>.add(new Dog())



printAnimal(List<Dog> dogs) {

==

2

printAnimal(List<Cat> cats) {

==

2

1
.
.
.

Code
Duplicacy

```
PrintAnimals( List<Animal> animals ){
    for(Animal animal : animals){
        Print(animal.name);
    }
}
```

2

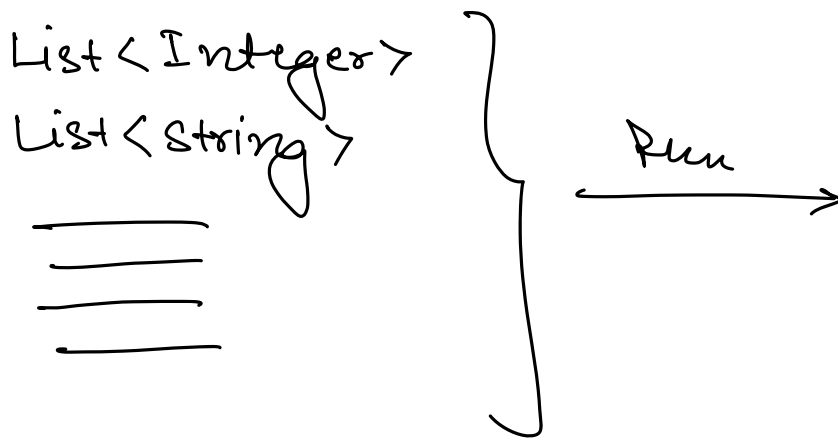
Solⁿ: Instead of accepting only Animal, start accepting Animal + all the child classes of Animal.

Animal & any of their
↓ child.

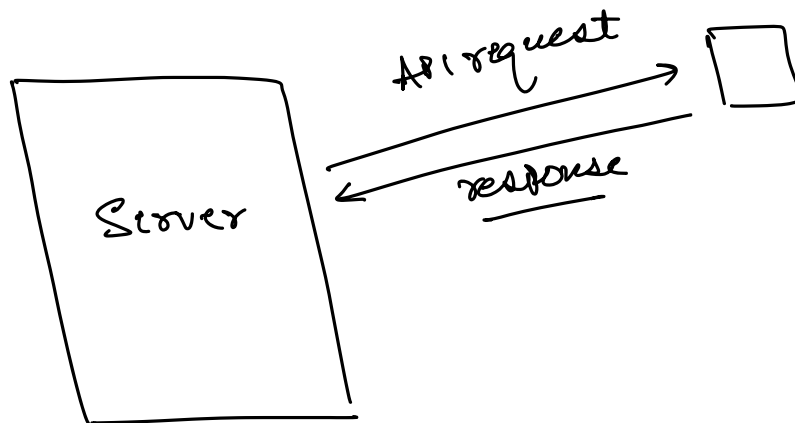
```
PrintAnimals( List<? extends Animal> animals ){
    for(Animal animal : animals){
        Print(animal.name);
    }
}
```

2

Type Erasure.



⇒ Runtime ⇒ List<Object>



restTemplate.get(URL, Product.class) ✓

Product {

id
name
desc
Price

}

⇒ JSON.

{
 "id" : "1"
 "name" : "Iphone",
 "desc" : "Best iphone",
 "price" : "100,000"
}

}

List<Product>.

restTemplate.get(URL, List<Product>.class)

```
[
  {
    "id": "1",
    "name": "Iphone",
    "desc": "Best iphone",
    "price": "100.000"
  },
  {
    "id": "2",
    "name": "Iphone",
    "desc": "Best iphone",
    "price": "100.000"
  },
  {
    "id": "3",
    "name": "Iphone",
    "desc": "Best iphone",
    "price": "100.000"
  },
]
```

⇒ List<Product>.

]]